ITA14 ETHICAL HACKING
ASSIGNMENT 3 – CO3

Mode: Pair Work – 2 Students per Group

Duration: 1 Week

Total Marks: 50

Name: C Sangita

Reg No: 192373059

Pair-Work Principle

Both students are responsible for understanding the complete assignment. However, each question contains clearly identified individual responsibilities so that the contribution and understanding of each student can be independently verified.

Student A and Student B must not divide the assignment question-wise. Both students must participate in every question.

The individual responsibilities intentionally alternate across questions so that both students demonstrate evidence of:

Q1 – Investigate Requirements and Create the Scan-Data Architecture — 6 Marks

Shared Group Work

Together, the pair must:

- Together, translate the authorized-assessment scenario into service-scanning and enumeration requirements.
- Identify at least four scan datasets, such as Hosts, Ports, Services, OS/Versions and Enumeration findings (sources: *nmap -sn*, *nmap -sS / -sU*, *nmap -sV*, *nmap -O*, *enum4linux*, *nbtscan*, *adb*).
- Create a schema and relationship plan showing the columns, data types and key fields (such as IP address and port) for each dataset, recorded as CSV header files and a relationship diagram.
- Formulate enquiry questions and define acceptance criteria for correctness, consistency, traceability and reproducibility of the scan data.

Student A – Host and Port Data Design

Student A must individually:

- Lead the requirements analysis and design the Hosts and Ports schemas, including fields, data types and key fields.
- Prepare the initial requirements-to-column mapping and justify the choice of scan-data structure.

Student B – Service Relationship and Enquiry Design

Student B must individually:

- Lead the relationship map for Services, OS/Versions and Enumeration findings (Windows, Symbian, Java OS, Android, NetWare); identify the key fields that link them to hosts and ports.
- Formulate at least four testable enquiry questions and define acceptance criteria for the scan data. Each question must be answerable with a Kali command or shell pipeline.

Individual Verification

Faculty may ask Student A to explain the host and port data design, and Student B to explain the service relationships and enquiry questions.

Answer Student A:

INTRODUCTION

Ethical hacking is the process of identifying security weaknesses in computer systems, networks and applications with proper authorization. Network scanning and enumeration are important activities used by security analysts to identify active hosts, open ports, running services, operating systems and possible security exposures.

During an authorized security assessment, scanning tools such as Nmap, fping, enum4linux and other enumeration utilities generate large amounts of data. This information is often stored in different formats, which makes analysis and reporting difficult.

This assignment focuses on developing an automated scan-data preparation and processing pipeline using Kali Linux, Bash scripting, AWK, SED, GREP and other Linux utilities.

The proposed pipeline performs the following operations:

1. Scan-data collection.
2. Data validation.
3. Data cleaning and correction.
4. Data transformation.
5. Dataset merging.
6. Automated file reading and writing.
7. Reporting and performance evaluation.

The assignment is completed through pair work. Both students participate in all questions, while Student A takes individual responsibility for host and port data design, scan-data construction, cleaning, transformation, host-service integration, file import, validation and reporting.

OBJECTIVES

1. To understand network scanning and enumeration.
2. To design structured CSV datasets for scan results.
3. To collect host, port and service information using Kali Linux.
4. To identify missing, duplicate and inconsistent scan records.
5. To implement automated data-cleaning operations.
6. To transform scan data between wide and long formats.
7. To integrate multiple scan datasets using appropriate join operations.
8. To develop reusable Bash functions.
9. To generate accurate security assessment reports.
10. To evaluate the efficiency, correctness and reproducibility of the pipeline.

1.1 Requirements Analysis

The first stage is to identify the requirements of the authorized network assessment.

The main requirement is to collect and organize information about network hosts, open ports, running services and operating systems.

The proposed assessment uses an authorized private laboratory network.

Example laboratory network: 192.168.56.0/24

The following requirements are identified:

1. Identify all active hosts in the authorized network.
2. Identify TCP and UDP open ports.
3. Detect services running on open ports.
4. Identify operating system information.
5. Store scan results in separate CSV datasets.
6. Establish relationships between hosts, ports and services.
7. Maintain data accuracy and consistency.
8. Support future scanning and comparison.

1.2 Identification of Scan Datasets

Four major datasets are designed.

Dataset 1: Hosts

The Hosts dataset stores information about active devices discovered during network scanning.

CSV file: hosts.csv

Column	Data Type	Description
host_id	Integer	Unique host identifier
ip_address	String	IP address of the device
hostname	String	Device hostname
mac_address	String	MAC address
status	String	Host status
scan_date	String	Date of scanning

CSV Header:

host_id,ip_address,hostname,mac_address,status,scan_date

Sample Data:

1,192.168.56.101,server01,08:00:27:AA:BB:01,up,2026-11-05

2,192.168.56.102,client01,08:00:27:AA:BB:02,up,2026-11-05

3,192.168.56.103,android01,08:00:27:AA:BB:03,up,2026-11-05

Dataset 2: Ports

The Ports dataset stores information about open, closed and filtered ports.

CSV file: ports.csv

Column	Data Type	Description
ip_address	String	Host IP address
port	Integer	Port number
protocol	String	TCP or UDP
state	String	Open, closed or filtered
scan_type	String	Scan method

CSV Header:

ip_address,port,protocol,state,scan_type

Sample Data:

192.168.56.101,22,tcp,open,SYN

192.168.56.101,80,tcp,open,SYN

192.168.56.102,443,tcp,open,CONNECT

192.168.56.103,5555,tcp,open,SYN

Dataset 3: Services

The Services dataset identifies applications and services running on the discovered ports.

CSV file: services.csv

ip_address,port,protocol,service_name,service_version

Sample Data:

192.168.56.101,22,tcp,ssh,OpenSSH

192.168.56.101,80,tcp,http,Apache

192.168.56.102,443,tcp,https,nginx

192.168.56.103,5555,tcp,adb,Android Debug Bridge

Dataset 4: OS and Enumeration

This dataset stores operating system details and enumeration findings.

CSV file: os_enum.csv

ip_address,os_name,os_version,device_type,enumeration_source,finding

Sample Data:

192.168.56.101,Linux,Ubuntu,Server,Nmap,Operating system detected

192.168.56.102,Windows,10,Workstation,nbtscan,NetBIOS information detected

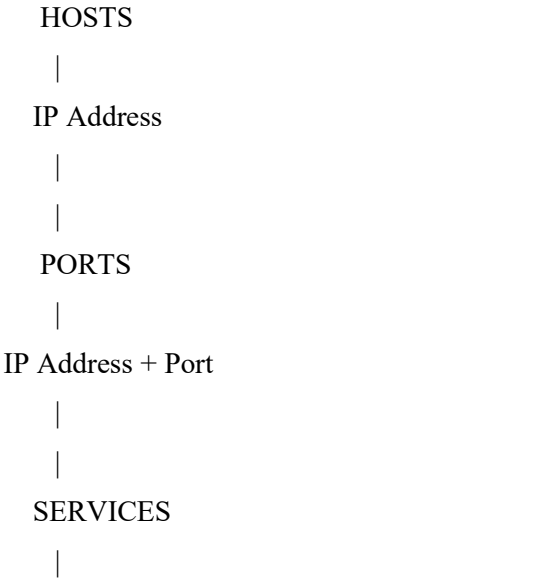
192.168.56.103,Android,14,Mobile,ADB,Device properties collected

1.3 Relationship Design

The datasets are connected using IP address and port number.

- IP address connects Hosts with Ports.
- IP address and port connect Ports with Services.
- IP address connects Hosts with OS and Enumeration findings.

Relationship Diagram



|

OS AND ENUMERATION

|

IP Address

The IP address acts as the primary linking field for host-related records.

For TCP and UDP services, the complete service key should include IP address, port and protocol.

1.4 Requirements-to-Column Mapping

Requirement	Dataset	Required Columns
Discover active devices	Hosts	ip_address, status
Identify open ports	Ports	ip_address, port, state
Detect services	Services	service_name, service_version
Identify operating system	OS	os_name, os_version
Record enumeration	Enumeration	enumeration_source, finding

1.5 Enquiry Questions

The following questions can be answered using Kali Linux commands.

Question 1: Which hosts are active in the network?

nmap -sn 192.168.56.0/24

Question 2: Which hosts have open TCP ports?

nmap -sS 192.168.56.0/24

Question 3: Which services are running on open ports?

nmap -sV 192.168.56.0/24

Question 4: What operating systems are detected?

sudo nmap -O 192.168.56.0/24

1.6 Acceptance Criteria

Criteria	Expected Result
Correctness	IP addresses and port numbers must be valid
Consistency	Column names and service names must follow standard formats
Traceability	Every record must contain its source information
Reproducibility	Repeated processing of the same input must produce identical output
Completeness	Required columns must be available
Uniqueness	Duplicate host-port-protocol records must be identified

1.7 Student A – Individual Responsibility

As Student A, I lead the requirements analysis and design of the Hosts and Ports datasets.

I selected IP address as the main linking field because multiple ports and services can belong to the same host. The port number identifies the particular network service endpoint.

I selected CSV format because it is simple, readable, portable and compatible with Linux command-line processing tools.

Result:

The scan-data architecture was designed successfully with four datasets. The Hosts and Ports schemas provide a structured foundation for collecting and organizing network scanning results.

Q2 – Create and Validate the Source Scan Datasets — 7 Marks

Shared Group Work

Together, the pair must:

- Together, construct realistic source scan datasets (hosts, ports, services and versions) using Kali tools (ping sweep with *nmap -sn* / *fping -a -g*; port scans with *nmap -sT, -sS, -sU*; service detection with *nmap -sV*; output saved with *-oX / -oG*) and create an appropriate CSV table for each.
- Introduce controlled issues such as missing values, duplicate host-port records, inconsistent service names and out-of-scope addresses (using *sed* and *awk* on a copy of the CSV).
- Develop validation and standardization checks before the datasets are merged.

Student A – Scan-Data Construction and Defect Injection

Student A must individually:

- Construct the Hosts and Ports/Services CSV tables and add controlled test defects.
- Explain the intended schema and the defects introduced for testing.

Student B – Scan-Data Validation and Standardization

Student B must individually:

- Construct the Services, OS/Versions and Enumeration CSV tables (*nmap -O*, *enum4linux -a* and *nbtscan* for Windows, *adb shell getprop* for Android, *rmi-dumpregistry* for Java, *ncp-serverinfo* for NetWare, *sdptool browse* for Symbian-type Bluetooth devices).
- Implement validation and standardization checks for required columns, valid IP and port ranges and consistent service names using *awk*, *grep -E*, *grepcidr* and a service-name lookup file.

Joint Design Question

Together answer:

If you were designing a scan-data preparation and validation pipeline for a large authorized assessment, which validation checks would you apply first, and why?

Both students must be able to defend the answer.

Answer Student A:

2.1 Scan-Data Construction

The second question focuses on generating scan datasets using Kali Linux tools.

The pair creates sample datasets representing hosts, ports, services and operating systems.

Step 1: Host Discovery

Nmap is used to identify active hosts.

```
nmap -sn 192.168.56.0/24 -oG hosts.gnmap
```

The output can be inspected using:

```
cat hosts.gnmap
```

Extract the active IP addresses:

```
awk '/Status: Up/ {print $2}' hosts.gnmap > active_hosts.txt
```

Step 2: TCP Port Scanning

```
sudo nmap -sS -p 22,80,443,445,3306,5555 \
192.168.56.101-103 -oX ports.xml
```

This command identifies the TCP port states of the authorized laboratory hosts.

Step 3: UDP Port Scanning

sudo nmap -sU -p 53,123,161 192.168.56.101-103

This command checks selected UDP ports.

Step 4: Service Detection

nmap -sV -p 22,80,443,445,3306,5555 \

192.168.56.101-103 -oX services.xml

This command identifies service names and available version information.

Step 5: Operating System Detection

sudo nmap -O 192.168.56.101-103 -oX os.xml

This command attempts to identify operating systems.

2.2 Creating Hosts CSV

Create the file:

nano hosts.csv

Enter:

host_id,ip_address,hostname,mac_address,status

1,192.168.56.101,server01,08:00:27:AA:BB:01,up

2,192.168.56.102,client01,08:00:27:AA:BB:02,up

3,192.168.56.103,android01,08:00:27:AA:BB:03,up

2.3 Creating Ports CSV

nano ports.csv

Enter:

ip_address,port,protocol,state

192.168.56.101,22,tcp,open

192.168.56.101,80,tcp,open

192.168.56.102,443,tcp,open

192.168.56.103,5555,tcp,open

2.4 Creating Services CSV

nano services.csv

Enter:

ip_address,port,protocol,service_name,service_version

192.168.56.101,22,tcp,ssh,OpenSSH

192.168.56.101,80,tcp,www,Apache

192.168.56.102,443,tcp,https,nginx

192.168.56.103,5555,tcp,adb,Android Debug Bridge

2.5 Introducing Controlled Defects

To test the validation process, controlled errors are introduced into a copy of the original data.

cp ports.csv ports_test.csv

Introduce a duplicate record:

echo "192.168.56.101,22,tcp,open" >> ports_test.csv

Introduce a missing value:

echo "192.168.56.102,,tcp,open" >> ports_test.csv

Introduce an invalid port:

echo "192.168.56.101,70000,tcp,open" >> ports_test.csv

Introduce an out-of-scope IP address:

echo "10.10.10.10,80,tcp,open" >> ports_test.csv

These defects are introduced only for testing the data-validation pipeline.

2.6 Validation Checks

Check 1: Missing Values

```
awk -F',' 'NR>1 {
for(i=1;i<=NF;i++)
if($i=="") print "Missing value at line",NR
}' ports_test.csv
```

Check 2: Duplicate Records

```
tail -n +2 ports_test.csv | sort | uniq -d
```

Check 3: Invalid Port Numbers

```
awk -F',' 'NR>1 && ($2<1 || $2>65535) {
print "Invalid port:",$2
}' ports_test.csv
```

Check 4: IP Address Format

```
grep -E '^[0-9]{1,3}(\.[0-9]{1,3}){3}$' ports_test.csv
```

This checks the basic IPv4 pattern. A separate numeric range check is also necessary.

Check 5: Standardizing Service Names

Create a lookup file:

nano service_map.txt

Enter:

www http

HTTP http

http-alt http

secure-http https

This file maps different service-name variations to standard names.

2.7 Student A – Individual Responsibility

My individual responsibility is to construct the Hosts and Ports CSV tables and introduce controlled test defects.

The intended schema separates host information from port information. This prevents unnecessary repetition of host details when a device has multiple open ports.

The introduced defects include:

1. Duplicate host-port records.
2. Missing port numbers.

3. Invalid port numbers.
4. Out-of-scope IP addresses.

These defects are useful for testing the correctness of the cleaning pipeline.

2.8 Joint Design Question

If you were designing a scan-data preparation and validation pipeline for a large authorized assessment, which validation checks would you apply first, and why?

The validation checks should be performed in the following order:

1. File existence and header validation.
2. Required-column validation.
3. Missing-value detection.
4. IP address validation.
5. Port-range validation.
6. Protocol and service-name standardization.
7. Duplicate-record detection.
8. Scope validation.
9. Relationship validation.

The first priority is to verify the file structure because later processing depends on correct columns. Invalid IP addresses and port numbers must be identified before merging datasets.

Scope validation is also important because unauthorized or unrelated systems must not be included in the assessment.

Result:

The source datasets were created and controlled defects were introduced. Validation commands were designed to detect missing values, duplicate records, invalid ports and inconsistent information.

Q3 – Build a Scan-Data Editing and Cleaning Module — 7 Marks

Shared Group Work

Together, the pair must:

- Together, develop reusable shell operations (Bash functions using *sed*, *awk*, *sort -u*) for missing values, inconsistent service names and duplicate host-port records.
- Show before-and-after data (*diff*, *wc -l*, *head*) and test valid, invalid and boundary cases (ports 0, 1, 65535, 65536; addresses 192.168.56.255 and 192.168.56.256).
- Distinguish records that can be corrected automatically from records that require manual review.

Student A – Scan-Data Cleaning and Correction Rules

Student A must individually:

- Lead cleaning operations for missing values, duplicate host-port records and malformed addresses.
- Explain the rules used for each automatic correction.

Student B – Service-Name Standardization and Test Validation

Student B must individually:

- Lead the rules for inconsistent service names (lookup file mapping variants such as www, HTTP, http-alt to one standard name) and manual-review flags; design verification tests.

- Verify that cleaning does not silently remove meaningful scan findings (record counts; every removed row must appear in a rejected or review file).

Joint Conclusion

Together answer:

Why is automated scan-data cleaning alone insufficient to guarantee the accuracy of an assessment's findings?

Both students must be able to justify their answers with suitable examples from their data-cleaning and validation tasks.

Answer Student A:

3.1 Introduction

Scan results may contain duplicate records, missing values, invalid IP addresses and inconsistent service names.

Data cleaning is necessary to improve the quality of scan information before analysis and reporting.

The cleaning module uses Bash, AWK, SED and SORT utilities.

3.2 Missing-Value Detection

The following command identifies missing values.

```
awk -F',' 'NR>1 {
for(i=1;i<=NF;i++)
if($i=="")
print "Missing value in row",NR
}' ports_test.csv
```

Purpose: Identifies records that contain empty fields.

Missing values should not be replaced with invented information. They should be marked for manual review when the original scan output cannot establish the correct value.

3.3 Duplicate Record Removal

```
head -n 1 ports_test.csv > ports_clean.csv

tail -n +2 ports_test.csv | sort -u >> ports_clean.csv
```

This operation retains the header and removes identical duplicate data rows.

However, records with the same IP and port but different protocols must not be treated as identical.

3.4 Invalid IP Address Detection

The following command checks whether the IP address contains four numeric octets.

```
awk -F',' 'NR>1 {
split($1,a,".")
if(length(a)!=4 ||
a[1]>255 || a[2]>255 ||
a[3]>255 || a[4]>255)
print "Invalid IP:",$1
}' ports_test.csv
```

A complete validation should also check that each octet contains only numeric digits and that no octet is empty.

3.5 Invalid Port Detection

```
awk -F',' 'NR>1 {
if($2<1 || $2>65535)
print "Invalid port:",$2
}' ports_test.csv
```

Valid TCP and UDP port numbers range from 1 to 65535.

Port 0 is reserved and should be rejected as an ordinary service port.

3.6 Service-Name Standardization

Create a service lookup file:

```
nano service_map.txt
```

Enter:

```
www http
HTTP http
http-alt http
secure-http https
```

A standardization operation can be implemented using AWK:

```
awk -F',' 'BEGIN{OFS=","}
FNR==NR {
map[$1]=$2
next
}
FNR==1 {
print
next
}
{
if($4 in map)
$4=map[$4]
print
}' service_map.txt services.csv > services_clean.csv
```

The command replaces recognized service-name variations with standard names.

3.7 Reusable Cleaning Function

The following Bash function removes exact duplicate records while preserving the header.

```
clean_duplicates() {

    input="$1"
    output="$2"

    if [ ! -f "$input" ]; then
```

```
        echo "Error: Input file not found" >&2
        return 1
    fi

    head -n 1 "$input" > "$output"

    tail -n +2 "$input" | sort -u >> "$output"

    return 0
}
```

Usage:

clean_duplicates ports_test.csv ports_clean.csv

3.8 Before-and-After Comparison

Before cleaning:

wc -l ports_test.csv

After cleaning:

wc -l ports_clean.csv

Display differences:

diff -u ports_test.csv ports_clean.csv

Display the cleaned records:

head ports_clean.csv

The line counts can be used to identify whether duplicate records were removed.

Every removed record must be accounted for in a duplicate or rejection report. A production cleaning pipeline should preserve the original input file.

3.9 Boundary Test Cases

Test Case	Input	Expected Result
Port lower boundary	0	Invalid
Port minimum	1	Valid
Port maximum	65535	Valid
Port upper boundary	65536	Invalid
Valid IP boundary	192.168.56.255	Valid
Invalid IP boundary	192.168.56.256	Invalid

These test cases verify that the cleaning module correctly handles boundary values.

3.10 Automatic Correction and Manual Review

Issue	Action
Exact duplicate row	Automatic removal with audit record
Known service variation	Automatic standardization
Missing port number	Manual review
Invalid IP address	Reject or manual review

Unknown service name	Manual review
Missing OS version	Manual review

Automatic cleaning is suitable only when the correct value is known with confidence.

3.11 Student A – Individual Responsibility

As Student A, I lead the cleaning operations for missing values, duplicate host-port records and malformed IP addresses.

My main objective is to ensure that the cleaning process does not destroy meaningful scan findings.

I use AWK for validation, SORT for duplicate identification, SED for text correction and DIFF for comparing original and cleaned files.

3.12 Joint Conclusion

Why is automated scan-data cleaning alone insufficient to guarantee the accuracy of an assessment's findings?

Automated cleaning alone cannot guarantee complete accuracy because scanning data may contain incomplete or misleading information.

For example:

1. An unknown service name may be incorrectly replaced with another service.
2. A missing port number cannot be determined automatically.
3. A filtered port may be incorrectly interpreted as closed.
4. An operating system detection result may be uncertain.
5. Duplicate-looking records may represent different protocols or scan sessions.

Therefore, automated cleaning must be combined with manual verification, original scan-output review and validation.

Result:

A reusable cleaning process was designed to identify missing values, duplicate records, malformed IP addresses and inconsistent service names.

Q4 – Create a Scan-Result Reshaping Pipeline Using Melting and Casting — 7 Marks

Shared Group Work

Together, the pair must:

- Together, create a wide-format scan dataset (for example hosts as rows and ports as columns, built from *nmap -p 21,22,23,25,80,139,443,445,3306,5555 -oG* output) and reshape it into long format (host, port, state) using *awk*.
- Create a cast operation (long to wide) for a host-by-service or subnet-by-service summary using *awk* arrays, *datamash crosstab* or *sort | uniq -c*.
- Identify the identifiers, dimensions and measured variables and validate that the reshape preserves the information (row counts and checksum of the round-trip output).

Student A – Scan-Result Reshaping and Transformation

Student A must individually:

- Implement the wide-to-long transformation and identify the identifier and measured columns.
- Explain why the selected long format suits the required analysis.

Student B – Summary Casting and Result Validation

Student B must individually:

- Create the cast/summary output (such as open-ports-per-host) and compare its totals with the source data.
- Explain how the validation demonstrates that the transformation preserved the scan information.

Joint Conclusion

Together answer:

- Why does a more detailed scan result not always produce a clearer or more useful assessment summary?
- Both students must be able to justify their decision using theoretical complexity (a full *-p-* scan probes 65,535 ports per host per protocol; a long table grows as hosts x ports) and experimental results (*time nmap*, *wc -l*).

Answer Student A:

4.1 Introduction

Scan results can be stored in wide format or long format.
Wide format stores multiple port values in separate columns.
Long format stores each host-port combination as an individual row.
Reshaping makes scan information easier to analyze, filter and summarize.

4.2 Creating Wide-Format Dataset

Create a file:
nano wide.csv
Enter:

```
ip_address,22,80,443,445,3306,5555
192.168.56.101,open,open,closed,open,closed,closed
192.168.56.102,closed,closed,open,open,open,closed
192.168.56.103,closed,closed,closed,closed,closed,open
```

Here:

- Rows represent hosts.
- Columns represent ports.
- Cell values represent port states.

4.3 Wide-to-Long Transformation (Melting)

The wide-format dataset is converted into long format using AWK.

```
awk -F',' 'BEGIN {
OFS=","
print "ip_address,port,state"
}
NR>1 {
for(i=2;i<=NF;i++)
print $1,$(i-1),$i
}' wide.csv > long.csv
```

Output format:

```
ip_address,port,state
192.168.56.101,22,open
192.168.56.101,80,open
192.168.56.101,443,closed
192.168.56.101,445,open
192.168.56.101,3306,closed
192.168.56.101,5555,closed
192.168.56.102,22,closed
```

192.168.56.102,80,closed
192.168.56.102,443,open
The same process continues for all hosts and ports.

4.4 Identification of Variables

Variable	Type	Description
ip_address	Identifier	Identifies each host
port	Dimension	Identifies the scanned port
state	Measured variable	Stores the port state

Long format is useful because each port can be filtered, grouped and counted independently.

4.5 Casting – Long to Summary Format

The long-format dataset can be summarized to count open ports per host.

```
awk -F',' 'NR>1 && $3=="open" {
count[$1]++
}
END {
for(ip in count)
print ip,count[ip]
}' long.csv > open_ports_summary.csv
```

The summary contains:
192.168.56.101 3
192.168.56.102 3
192.168.56.103 1
These values are based on the illustrative wide-format dataset.

4.6 Validating the Transformation

Count the original hosts:
tail -n +2 wide.csv | wc -l
Count the number of port columns:
head -n 1 wide.csv | awk -F',' '{print NF-1}'
Count the long-format records:
tail -n +2 long.csv | wc -l
For the sample dataset:

- Number of hosts = 3
- Number of port columns = 6
- Expected long-format records = 3 × 6 = 18

Therefore:
Expected records = 18
Actual records = 18
This verifies that each host-port combination was represented.

4.7 Round-Trip Validation

The long-format data can be reconstructed into wide format.
The reconstruction should preserve the original host, port and state values.
Use:
sha256sum wide.csv
sha256sum wide_reconstructed.csv
The hashes should match only when the reconstructed file has exactly the same content and ordering asthe original.
If the row order differs, sort both datasets using a consistent key before comparison.

4.8 Student A – Individual Responsibility

My individual responsibility is to implement the wide-to-long transformation.

I use AWK because it can process CSV-like records line by line and efficiently transform columns intorows.

The long format is selected because it simplifies:

1. Filtering open ports.
2. Counting services.
3. Comparing hosts.
4. Grouping scan results.
5. Integrating different scan datasets.

4.9 Joint Conclusion

Question 1: Why does a more detailed scan result not always produce a clearer assessment summary?

A detailed scan produces more information, but it does not always make the final report easier to understand.

For example, a full TCP port scan checks up to 65,535 TCP port numbers per host. A UDP full-port scan can add another 65,535 port numbers.

A long-format table can grow approximately as:

Number of hosts × Number of scanned ports.

Large datasets require more storage, processing and analysis.

Therefore, management reports should focus on important open services, exposed systems and relevantsecurity risks instead of displaying every scanning detail.

Question 2: What is the theoretical and experimental complexity?

Theoretical complexity:

Long-format records = Number of hosts × Number of ports

For 50 hosts and 100 ports:

50 × 100 = 5,000 records

For 5,000 hosts and 1,000 ports:

5,000 × 1,000 = 5,000,000 records

Experimental performance can be measured using:

time awk -F',' 'NR>1 {print $0}' wide.csv

The elapsed time and generated record count help evaluate transformation performance.

Result:

The wide-format dataset was transformed into long format using AWK. An open-port summary was also designed, and record-count validation was used to verify the transformation.

Q5 – Design a Multi-Stage Scan-Data Merging Strategy — 6 Marks

Shared Group Work

Together, the pair must:

- Together, combine the Hosts, Ports, Services, OS/Versions and Enumeration datasets into a single master scan table using *join*, *awk* arrays and *sort*.
- Identify the keys (such as IP address and port), select suitable merge types (*join* inner, *-a1* left, *-a1 -a2* full outer, *-v* unmatched) and test at least two alternatives.
- Detect unmatched records (*join -v*, *comm -23*), duplicated matches (*sort* | *uniq -d*) and unexpected row multiplication after merging (*wc -l* before and after).

Student A – Host–Service Integration and Join Validation

Student A must individually:

- Lead the Services-to-Hosts and Ports-to-Hosts joins; document the key choices.
- Check the record counts and ensure the joins do not create unintended row multiplication.

Student B – Version–Enumeration Integration and Exception Analysis

Student B must individually:

- Lead the OS/Versions and Enumeration integration (Windows SMB/NetBIOS, Android ADB, Java RMI, NetWare NCP findings); compare alternative join types.
- Help verify the final master scan table and document exceptions.

Cross-Understanding Requirement

Student B must be able to explain:

Why can different join types produce different record counts, and how can unmatched hosts or services be detected?

Student A must be able to explain:

What experimental evidence demonstrates that the Services-to-Hosts and Ports-to-Hosts joins are correct?

This deliberately tests understanding across the assigned responsibilities.

Answer Student A:

.1 Introduction

A network assessment produces multiple datasets. These datasets must be integrated to create a complete view of each discovered host.

The merging process combines:

1. Hosts.
2. Ports.
3. Services.
4. OS and Versions.
5. Enumeration findings.

The main purpose is to produce a master scan table.

5.2 Identifying Merge Keys

The following keys are selected:

Dataset	Merge Key
Hosts	ip_address
Ports	ip_address
Services	ip_address + port + protocol
OS	ip_address
Enumeration	ip_address

For service integration, IP address, port and protocol are used together to avoid incorrect matching.

5.3 Sorting the Datasets

The Linux JOIN command requires input files to be sorted using the selected joining field.

Example:

sort -t',' -k1,1 hosts.csv > hosts_sorted.csv

Similarly:

sort -t',' -k1,1 ports.csv > ports_sorted.csv

sort -t',' -k1,1 services.csv > services_sorted.csv

Headers should be handled separately in a real CSV-processing pipeline.

5.4 Host and Port Integration

Student A is responsible for integrating Hosts and Ports.

Example:

```
join -t',' -1 2 -2 1 \
hosts_sorted.csv ports_sorted.csv \
> host_ports.csv
```

Here:

- Hosts column 2 contains the IP address.
- Ports column 1 contains the IP address.

The join operation matches records belonging to the same host.

For actual CSV files, headers should be removed before joining and restored afterward.

5.5 Services-to-Hosts Integration

The service information is integrated with the host information using IP address.

Example:

```
sort -t',' -k1,1 services.csv > services_sorted.csv
join -t',' -1 2 -2 1 \
hosts_sorted.csv services_sorted.csv \
> host_services.csv
```

This creates a combined view of host and service information.

5.6 Types of Join Operations

Inner Join

```
join -t',' file1.csv file2.csv
```

Returns only matching records.

Left Outer Join

```
join -t',' -a1 file1.csv file2.csv
```

Preserves all records from the first file.

Full Outer Join

```
join -t',' -a1 -a2 file1.csv file2.csv
```

Preserves records from both files, including unmatched records.

Unmatched Records

```
join -t',' -v1 file1.csv file2.csv
```

Displays records from the first file that do not have a match.

5.7 Detecting Duplicate Records

```
sort host_ports.csv | uniq -d
```

This identifies exact duplicate lines.

To identify repeated host-port-protocol combinations:

```
awk -F',' 'NR>1 {
key=$1 FS $2 FS $3
count[key]++
```

```
}
END {
for(k in count)
if(count[k]>1)
print "Duplicate:",k
}' ports.csv
```

5.8 Checking Record Multiplication

Before merging:

```
wc -l hosts.csv ports.csv services.csv
```

After merging:

```
wc -l host_ports.csv
```

The merged record count should be explained by the relationship between hosts and their ports.

One host may legitimately produce several merged records because it can have multiple open ports.

Unexpected multiplication may occur when the join key is not unique or when duplicate records exist.

5.9 Student A – Individual Responsibility

My individual responsibility is to lead the Services-to-Hosts and Ports-to-Hosts integration.

I selected IP address as the host relationship key and IP address, port and protocol as the service endpoint key.

I validate the joins by checking:

1. The original dataset record counts.
2. The number of matching records.
3. The number of unmatched records.
4. Duplicate host-port combinations.
5. Unexpected record multiplication.

5.10 Cross-Understanding Requirement

What experimental evidence demonstrates that the Services-to-Hosts and Ports-to-Hosts joins are correct?

The following evidence is used:

- Matching IP addresses between the datasets.
- Correct IP-port-protocol relationships.
- No unexpected duplicate records.
- Correct unmatched-record reports.
- Record-count comparison before and after merging.

A sample host with two open services should produce two corresponding service records in the merged output.

5.11 Joint Conclusion

Different join types produce different record counts because they handle unmatched records differently.

An inner join includes only matching records, while a left join preserves all records from the left dataset. A full outer join preserves records from both datasets.

Unmatched records can be detected using:

join -v1 file1.csv file2.csv

and:

join -v2 file1.csv file2.csv

Result:

A multi-stage merging strategy was designed using Linux JOIN and SORT commands. Host-service relationships and unmatched-record detection were included in the integration process.

Q6 – Create an Automated Scan-File Reading and Writing Pipeline — 7 Marks

Shared Group Work

Together, the pair must:

- Together, read source scan-output files (CSV or Nmap results exported to CSV from XML or grepable output using *xsltproc* / *awk*), check the required columns and import them into CSV tables.
- Create a naming and versioning strategy that avoids accidentally overwriting earlier scan results (for example subnet_scantype_YYYYMMDD-HHMMSS_vN.csv, *set -o noclobber*, *sha256sum* checksum file).

Student A – Scan-File Import and Schema Validation

Student A must individually:

- Implement scan-file import and schema/type checks before processing (header match, numeric port column, valid IP pattern).
- Demonstrate how the pipeline identifies missing required columns or unexpected formats.

Student B – Result Export, Versioning and Read-Back Validation

Student B must individually:

- Implement export of the cleaned, reshaped and master scan datasets; define the output naming and versioning.
- Compare the re-imported data with the written outputs (*diff*, *cmp*, *sha256sum -c*, *wc -l*).

Answer Student A:

6.1 Introduction

Scan tools generate results in different formats such as XML, grepable output and CSV.

An automated pipeline is required to read these files, verify their structure, process the data and export the final results.

The pipeline must also prevent accidental overwriting of previous scan files.

6.2 Reading Nmap XML Output

Nmap can save scan results in XML format.

nmap -sV 192.168.56.101-103 -oX scan.xml

The XML output can be converted into CSV using an appropriate XML parser or XSLT transformation.

For example:

xsltproc scan.xml -o scan.html

This produces an HTML report, not a CSV file. A dedicated XML-to-CSV transformation is required for CSV export.

A simple CSV can also be created from grepable Nmap output using AWK.

nmap -sV -oG scan.gnmap 192.168.56.101-103

Extract host and port information:

awk '

/Ports:/ {

```
ip=$2
n=split($0,a,"Ports: ")
if(n>1)
print ip "," a[2]
}' scan.gnmap
```

For production use, a proper parser should be used to handle multiple ports and fields correctly.

6.3 Import and Schema Validation

Create an import script:

nano import.sh

Enter:

```
#!/bin/bash

set -euo pipefail

INPUT="${1:-ports.csv}"

if [ ! -f "$INPUT" ]; then
   echo "Error: File not found" >&2
   exit 1
fi

EXPECTED="ip_address,port,protocol,state"

HEADER=$(head -n 1 "$INPUT")

if [ "$HEADER" != "$EXPECTED" ]; then
   echo "Error: Invalid CSV header" >&2
   exit 1
fi

echo "File imported successfully"
```

Run:

```
chmod +x import.sh
./import.sh ports.csv
```

6.4 Port and IP Validation

The following AWK operation checks port ranges.

```
awk -F',' 'NR>1 {
if($2 !~ /^[0-9]+$/ || $2<1 || $2>65535)
print "Invalid port in line",NR
}' ports.csv
```

IP validation:

```
awk -F',' 'NR>1 {
split($1,a,".")
if(length(a)!=4 ||
a[1]<0 || a[1]>255 ||
a[2]<0 || a[2]>255 ||
a[3]<0 || a[3]>255 ||
a[4]<0 || a[4]>255)
print "Invalid IP in line",NR
}' ports.csv
```

A production version should also check that every octet is numeric and non-empty.

6.5 File Naming and Versioning

A naming convention is used to prevent accidental overwriting.

Example:

192.168.56.0-24_tcp_20261105-130000_v1.csv

192.168.56.0-24_tcp_20261105-140000_v2.csv

192.168.56.0-24_udp_20261105-150000_v1.csv

The naming structure contains:

Network range.

Scan type.

Date and time.

Version number.

6.6 Preventing File Overwriting

Enable noclobber:

```
set -o noclobber
```

Create a versioned file:

```
OUTPUT="192.168.56.0-24_tcp_20261105-130000_v1.csv"

echo "ip_address,port,protocol,state" > "$OUTPUT"
```

If the file already exists, the shell prevents accidental overwriting.

6.7 Generating Checksums

```
sha256sum ports_clean.csv > ports_clean.sha256
```

Verify the file:

```
sha256sum -c ports_clean.sha256
```

The checksum helps verify that the file has not changed unexpectedly.

6.8 Read-Back Validation

Compare the exported and imported files.

```
cmp ports_clean.csv ports_export.csv
```

Check the line count:

```
wc -l ports_clean.csv ports_export.csv
```

Compare checksums:

```
sha256sum ports_clean.csv ports_export.csv
```

If the contents are identical, the checksum values should match.

6.9 Student A – Individual Responsibility

My individual responsibility is scan-file import and schema validation.

I developed an import process that verifies:

File existence.

Correct CSV header.

Required columns.

Numeric port values.

Valid IP addresses.

Expected input format.

The main objective is to prevent invalid source files from entering the scan-data processing pipeline.

Result:

An automated scan-file import and validation process was designed. File naming, versioning and checksum verification were included to protect the integrity of scan results.

DAY-4 DESIGN CHALLENGE

Q7 – Develop a Reusable Scanning and Enumeration Function Library — 10 Marks

The Day-4 problem must be solved collaboratively.

Neither student may work on this question independently and simply combine the answers later. The library is a Bash file (scanlib.sh, loaded with *source*) whose functions take the subnet, scan type and file names as arguments with default values, return status codes and write errors to stderr.

Stage 1 – Independent Function Design Proposal

Student A – Validation and Cleaning Function Design

Student A must individually:

- Lead the validation and cleaning functions for scan data; specify inputs, default arguments and return values.
- Test the functions independently and document their use.

Student B – Scan Transformation and File-Handling Function Design

Student B must individually:

- Lead the reshaping, merging and file-handling functions; include error handling (*set -euo pipefail*, *trap*, missing-file checks).
- Demonstrate reuse with a different subnet or scan session.

Each student's initial proposal must be retained in the submission.

No marks are deducted merely because an initial design is incorrect.

The purpose is to provide evidence of independent algorithmic thinking.

Stage 2 – Integrated Pipeline Implementation and Validation

Student A – Import, Validation and Cleaning Integration

Student A must individually:

- Integrate the import, validation and cleaning stages; record the input, rejected and cleaned record counts.
- Check that invalid inputs are handled consistently.

Student B – Transformation, Merging and Export Integration

Student B must individually:

- Integrate the reshaping, merging and export stages; record the merged and final record counts.
- Run the workflow again with the same inputs and check reproducibility (identical *sha256sum* of outputs).

Each student must document their implementation, test cases, record counts, and validation results.

Both students must demonstrate that the integrated pipeline operates correctly and produces consistent results.

The purpose is to assess independent implementation, integration accuracy, and reproducibility.

Stage 3 – Create and Validate a Reporting Scan Summary

Student A – Host and Service Reporting and Reconciliation

Student A must individually:

- Create per-host and per-service summary reports (*awk*, *sort* | *uniq -c*) and document how they are calculated.
- Verify the reported totals against the master scan table.

Student B – Vulnerability-Exposure Reporting and Management Report

Student B must individually:

- Create an exposure or service-version risk report (for example outdated services found with *nmap -sV*, *nmap --script vuln* or *searchsploit* on the lab targets) and implement the summary.
- Explain how the report addresses a management requirement.

Stage 4 – Stress-Test, Refine and Optimize the Scan Pipeline

Student A – Dataset Scaling and Baseline Benchmarking

Student A must individually:

- Generate or prepare scan datasets at the required scales (for example 50 and 5,000 host-port records) and benchmark the baseline (*time*, */usr/bin/time -v*).
- Record input size, elapsed time and correctness checks.

Student B – Bottleneck Analysis and Performance Optimization

Student B must individually:

- Analyse the measurements, identify a bottleneck and implement one refinement (for example replace a per-line *grep* loop with a single *awk* pass, or tune discovery with *nmap -T4 --min-rate* / *masscan*).
- Compare before-and-after performance and explain any limitations.

Stage 5 – Redesign the Pipeline Under a Changed Assessment Requirement

Student A – Schema Evolution and Architecture Redesign

Student A must individually:

- Analyse the impact on the schema, keys, validation and data retention. Propose revised data structures and rules for new scan sources such as UDP services (*nmap -sU*, key becomes IP + port + protocol) or newly discovered hosts (including Symbian, Java OS, Android and NetWare devices).
- Retain the impact analysis and explain the revised architecture.

Student B – Multi-Session Workflow Implementation and Change Validation

Student B must individually:

- Implement or modify the multi-session scan/merge workflow and test retained, updated, unmatched and newly discovered hosts (compare sessions with *ndiff* and *comm*).
- Present test evidence, compare with the original pipeline and identify trade-offs.

Stage 6 – Joint Evaluation

Together determine:

- Why the scan-data pipeline requires a structured sequence of import, validation, cleaning, reshaping, merging and export operations.
- How modular functions and systematic transformations improve the correctness, maintainability and efficiency of the scanning workflow.
- What additional costs or challenges arise from validation, error handling, repeated scanning and data integration.
- Under what conditions the proposed pipeline might become inefficient, unreliable or unsuitable for changing assessment requirements.

Conclude:

What characteristics should a security analyst consider when designing a reliable, efficient and maintainable scan-data pipeline for evolving networks?

Answer Student A:

7.1 Introduction

A reusable scanning and enumeration function library reduces repeated code and makes the assessment pipeline easier to maintain.
The library is created using Bash functions and stored in a file named:
scanlib.sh
The library contains reusable functions for:
Importing scan files.
Validating CSV files.
Cleaning duplicate records.
Checking port numbers.
Transforming scan results.
Merging datasets.
Exporting final results.
Generating reports.
The functions accept file names and other required parameters, return status codes and write error messages to standard error.

7.2 Stage 1 – Independent Function Design Proposal

Student A – Validation and Cleaning Function Design

My initial proposal focuses on creating reusable validation and cleaning functions.
The proposed functions are:

Function	Input	Purpose
check_file()	Filename	Checks file existence
validate_header()	Filename	Checks required columns
validate_ports()	Filename	Checks port ranges
validate_ip()	Filename	Checks IP addresses
clean_duplicates()	Input, Output	Removes exact duplicate rows
check_missing()	Filename	Detects missing fields

Function Design

1. check_file()
Input: File name.
Output: Success or failure status.

2. validate_header()
Input: CSV file and expected header.
Output: Valid or invalid schema.

3. validate_ports()
Input: Ports CSV.
Output: Invalid port records.

4. clean_duplicates()
Input: Source CSV and destination CSV.
Output: Duplicate-free CSV.

Initial Proposal Justification

The validation functions must run before cleaning because invalid records should be identified before modification.
The original scan file must be retained so that the cleaning process can be verified.

7.3 Stage 2 – Integrated Pipeline Implementation and Validation

Student A – Import, Validation and Cleaning Integration

Create the Bash library:
nano scanlib.sh
Enter the following implementation:

```
#!/bin/bash

check_file() {

   local file="$1"

   if [[ ! -f "$file" ]]; then
      echo "ERROR: File not found: $file" >&2
      return 1
   fi

   return 0
}


validate_header() {

   local file="$1"
   local expected="$2"

   check_file "$file" || return 1

   local actual
   actual=$(head -n 1 "$file")

   if [[ "$actual" != "$expected" ]]; then
      echo "ERROR: Invalid header" >&2
      return 1
   fi

   return 0
}


validate_ports() {

   local file="$1"

   check_file "$file" || return 1

   awk -F',' '
   NR>1 {
      if($2 !~ /^[0-9]+$/ || $2<1 || $2>65535) {
         print "Invalid port:",$2
         error=1
      }
   }
   END {
      exit error
   }' "$file"
```

```
}


clean_duplicates() {

   local input="$1"
   local output="$2"

   check_file "$input" || return 1

   head -n 1 "$input" > "$output"

   tail -n +2 "$input" |
   sort -u >> "$output"

   return 0
}


check_missing() {

   local file="$1"

   check_file "$file" || return 1

   awk -F',' '
   NR>1 {
      for(i=1;i<=NF;i++) {
         if($i=="") {
            print "Missing field at line",NR
            error=1
         }
      }
   }
   END {
      exit error
   }' "$file"
}
```

7.4 Loading the Library

The library can be loaded using:

source scanlib.sh

Validate the file:

validate_header ports.csv \
"ip_address,port,protocol,state"

Check the port values:

validate_ports ports.csv

Check missing values:

check_missing ports.csv

Remove duplicate records:

clean_duplicates ports.csv ports_clean.csv

7.5 Integrated Pipeline Execution

Create a file:

nano main.sh

Enter:

```
#!/bin/bash

set -euo pipefail

trap 'echo "Pipeline failed at line $LINENO" >&2' ERR

source ./scanlib.sh

INPUT="ports.csv"
OUTPUT="ports_clean.csv"

echo "Starting scan-data pipeline"

check_file "$INPUT"

validate_header "$INPUT" \
"ip_address,port,protocol,state"

validate_ports "$INPUT"

check_missing "$INPUT"

echo "Validation completed"

clean_duplicates "$INPUT" "$OUTPUT"

echo "Cleaning completed"

echo "Input records:"
tail -n +2 "$INPUT" | wc -l

echo "Cleaned records:"
tail -n +2 "$OUTPUT" | wc -l

echo "Pipeline completed"
```

Run:

```
chmod +x main.sh
./main.sh
```

This integrates file checking, schema validation, port validation, missing-value checking and duplicate cleaning.

7.6 Stage 2 – Test Cases

The following test cases are used to verify the integrated pipeline.

Test	Input	Expected Result
Valid file	Correct CSV	Accepted
Missing file	Unknown filename	Error

Invalid header	Incorrect columns	Rejected
Port 0	Invalid port	Rejected
Port 1	Valid port	Accepted
Port 65535	Valid port	Accepted
Port 65536	Invalid port	Rejected
Duplicate record	Repeated row	Duplicate identified
Missing value	Empty field	Flagged for review

Record Count Verification

wc -l ports.csv

wc -l ports_clean.csv

The record counts are compared to determine whether duplicates were removed.

The original, rejected and cleaned record counts should be documented.

7.7 Stage 3 – Create and Validate a Reporting Scan Summary

Student A – Host and Service Reporting and Reconciliation

My responsibility is to generate per-host and per-service summary reports.

1. Per-host summary

```
awk -F',' 'NR>1 {
count[$1]++
}
END {
for(ip in count)
print ip,count[ip]
}' ports_clean.csv > host_summary.txt
```

This counts the number of port records associated with each IP address.

2. Open ports per host

```
awk -F',' 'NR>1 && $4=="open" {
count[$1]++
}
END {
for(ip in count)
print ip,count[ip]
}' ports_clean.csv > open_ports_host.txt
```

3. Service summary

```
awk -F',' 'NR>1 {
count[$4]++
}
END {
for(service in count)
print service,count[service]
}' services.csv > service_summary.txt
```

4. Reconciliation

The total number of service records should match the sum of the individual service counts.

Check:

wc -l services.csv

cat service_summary.txt

The header should be excluded when comparing data-row totals.

Management Report

The host and service reports help administrators understand:

Number of discovered hosts.
Number of detected services.
Distribution of open ports.
Systems with multiple exposed services.
Devices requiring further security review.
These reports convert raw scanning data into information that can support security decisions.

7.8 Stage 4 – Stress-Test, Refine and Optimize the Scan Pipeline

Student A – Dataset Scaling and Baseline Benchmarking

The pipeline is tested using different dataset sizes.
Two test sizes are selected:
Small dataset: 50 host-port records.
Large dataset: 5,000 host-port records.
Generate a sample dataset:

```
echo "ip_address,port,protocol,state" > large.csv

for i in $(seq 1 5000)
do
   echo "192.168.56.$((i%250+1)),$((i%65535+1)),tcp,open" >> large.csv
done
```

The resulting file contains 5,000 data records and one header.

Benchmarking

Use:

```
time ./main.sh
```

For detailed performance information:

```
/usr/bin/time -v ./main.sh
```

The following measurements are recorded:

Measurement	Description
Input records	Number of source records
Output records	Number of cleaned records
Elapsed time	Total processing time
CPU usage	Processor utilization
Maximum memory	Peak memory consumption
Correctness	Validation and record-count checks

Actual timing values should be filled in after running the commands.

Student A – Individual Responsibility

My responsibility is to generate or prepare datasets at different scales and benchmark the baseline pipeline.
The main objective is to determine whether the pipeline can process larger scan datasets without losing records or producing incorrect results.

7.9 Stage 4 – Bottleneck Optimization

A possible optimization is to use one AWK pass instead of repeatedly processing the same file through separate commands.
Example:

```
awk -F',' '
NR==1 {
   print
   next
}
```

```
{
  if($2 ~ /^[0-9]+$/ && $2>=1 && $2<=65535)
    print
  else
    print > "rejected.csv"
}' large.csv > optimized.csv
```

This approach validates port values in a single pass.

Performance should be compared using:

```
time awk -F',' 'NR>1 {print $0}' large.csv
```

The optimization should be evaluated using both processing time and correctness.

7.10 Stage 5 – Redesign the Pipeline Under a Changed Assessment Requirement

Student A – Schema Evolution and Architecture Redesign

A security assessment may change when new devices, protocols or network services are discovered. The existing architecture must support these changes.

Change 1: UDP Services

Initially, the service relationship uses:

IP Address + Port

When UDP services are introduced, the key must become:

IP Address + Port + Protocol

The revised Ports CSV becomes:

```
ip_address,port,protocol,state,scan_type
```

Example:

```
192.168.56.101,53,udp,open,UDP
192.168.56.101,80,tcp,open,TCP
192.168.56.102,161,udp,open,UDP
```

This prevents TCP and UDP services using the same port number from being incorrectly combined.

Change 2: New Device Types

The assessment may identify additional devices such as:

Windows devices.

Android devices.

Java-based systems.

NetWare devices.

Symbian-type Bluetooth devices.

The OS and Enumeration dataset must support additional device information.

Revised schema:

```
ip_address,device_type,os_name,os_version,protocol,enumeration_source,finding
```

Change 3: New Validation Rules

The updated pipeline should:

Validate TCP and UDP protocols.

Validate IP addresses.

Validate port ranges.

Support multiple enumeration sources.

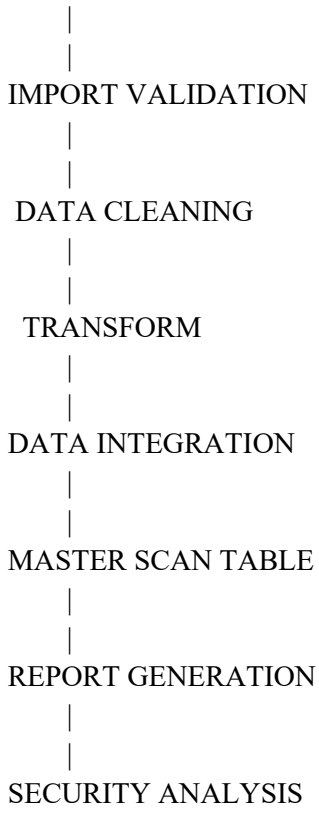
Preserve previous scan sessions.

Identify newly discovered devices.

Maintain traceability of findings.

Revised Architecture

```
        SCAN SOURCES
            |
            |
      DATA COLLECTION
```



Student A – Individual Responsibility

My responsibility is to analyse how the addition of UDP scanning and new device types affects the schema, keys and validation rules.

I recommend adding protocol as part of the service key and extending the OS and Enumeration datasets to store information from different device types.

7.11 Stage 5 – Multi-Session Workflow

A multi-session workflow helps compare network conditions across different scanning dates.

For example:

scan_session_1.csv

scan_session_2.csv

The Nmap difference utility can compare two scan sessions:

ndiff scan1.xml scan2.xml

The comm command can compare sorted host lists:

comm -23 hosts_session1.txt hosts_session2.txt

This identifies records present in the first session but not in the second.

Newly discovered records can be identified using:

comm -13 hosts_session1.txt hosts_session2.txt

The comparison helps identify:

Retained hosts.

Removed or unavailable hosts.

Newly discovered hosts.

Changed service information.

7.12 Stage 6 – Joint Evaluation

Question 1: Why does the scan-data pipeline require a structured sequence of import, validation, cleaning, reshaping, merging and export operations?

A structured sequence is necessary because each stage prepares the data for the next stage.

Import: Reads source scan files.

Validation: Checks whether the source information is correctly structured.

Cleaning: Identifies and handles duplicate, missing and invalid records.

Reshaping: Converts data into a format suitable for analysis.
Merging: Combines information from different scan datasets.
Export: Stores the final results in reusable files.
If these stages are performed without a proper sequence, invalid or incomplete records may affect the final assessment.

Question 2: How do modular functions improve correctness, maintainability and efficiency?

Modular functions divide the complete pipeline into smaller operations.
For example, validate_ports() checks port ranges, while clean_duplicates() removes duplicate rows.
The advantages include:
Reusability.
Easy debugging.
Better error handling.
Easier maintenance.
Reduced code repetition.
Consistent processing.
Improved testing.

Question 3: What additional costs or challenges arise?

The pipeline introduces certain costs and challenges:
Additional processing time for validation.
Storage requirements for original and cleaned datasets.
Complexity in merging different scan formats.
Manual verification of uncertain findings.
Maintenance of service-name lookup files.
Error handling for missing or corrupted files.
Repeated scanning and version management.
Handling large datasets efficiently.

Question 4: Under what conditions might the pipeline become inefficient or unreliable?

The pipeline may become inefficient when:
The number of hosts and ports becomes extremely large.
Multiple scanning sessions produce duplicate information.
Different tools generate incompatible formats.
Validation operations repeatedly process the same files.
Incorrect join keys cause unexpected record multiplication.
The assessment requirements change frequently.
Network scanning produces incomplete or unreliable results.
To improve reliability, the pipeline should use modular scripts, suitable data structures, consistent schemas and regular validation.

7.13 Final Joint Conclusion

A reliable scan-data pipeline must be accurate, efficient, reusable, scalable and maintainable.
Security analysts should consider the following characteristics:
Correct data collection.
Proper authorization and defined scanning scope.
Consistent dataset structures.
Valid IP addresses and port numbers.
Reliable duplicate detection.
Correct service and operating system identification.
Appropriate transformation and merging techniques.
Error handling and exception reporting.
Reproducible processing.
Secure storage and version management.

Efficient performance for large datasets.
Adaptability to new scanning requirements.
The proposed pipeline provides a systematic approach for converting raw network scan results into structured, validated and meaningful security assessment information.

Result:

A reusable Bash-based scan-data pipeline was designed with import, validation, cleaning, transformation, merging, reporting and performance-testing stages. The architecture also supports changes such as UDP scanning, new device types and multi-session comparisons.

OVERALL ASSIGNMENT CONCLUSION

This assignment demonstrates the use of ethical hacking tools and Linux-based data-processing utilities to create an automated scan-data preparation and analysis pipeline.
The assignment begins with the identification of network scanning requirements and the design of structured CSV datasets. Host discovery, port scanning, service detection and operating system identification are used to collect network information.
The collected data is then validated and cleaned using AWK, SED, SORT and GREP. Wide-format data is converted into long format, and different datasets are integrated using JOIN operations.
A reusable Bash function library is designed to automate the processing stages. The pipeline also includes file versioning, checksum verification, reporting and performance evaluation.
As Student A, I focused on host and port schema design, scan-data construction, defect injection, data cleaning, wide-to-long transformation, host-service integration, file import, schema validation, host reporting, benchmarking and schema evolution.
Overall, this assignment improved my understanding of network scanning, enumeration, data processing, Bash scripting and security assessment reporting.
The proposed solution provides a structured and reusable approach for processing scan information while maintaining correctness, traceability and reproducibility.

CONTRIBUTION RECORD

Each pair must complete the following table at the end of the submission. The record must reflect actual work completed. Primary contribution does not mean exclusive responsibility.

Task	Student A – Primary Contribution	Student B – Primary Contribution	Joint Work
Q1			
Q2			
Q3			
Q4			
Q5			

Q6			
Q7			

.